

La Regressione Logistica

Machine Learning IA5

Corso ITS Data Analyst



Cosa imparerai oggi

In questa lezione esploreremo uno degli algoritmi fondamentali del Machine Learning: la regressione logistica. Partiremo dalle basi per costruire una comprensione solida e pratica di questo potente strumento di classificazione.

01

Regressione vs Classificazione

Capire la differenza fondamentale tra problemi di regressione e classificazione, e quando usare ciascun approccio

03

La Funzione Sigmoidale

Come funziona la funzione sigmoide e perché è perfetta per trasformare valori in probabilità

05

Decision Boundary

Come interpretare il confine decisionale, sia nei casi lineari che non lineari

07

Gradient Descent

Come applicare l'algoritmo del gradient descent per ottimizzare i parametri del modello

02

Limiti della Regressione Lineare

Perché la regressione lineare non funziona per i problemi di classificazione e quali problemi genera

04

Costruzione del Modello

Come si costruisce il modello di regressione logistica combinando funzioni lineari e sigmoide

06

Funzione di Costo

La funzione di costo logaritmica specifica per la regressione logistica e perché è necessaria

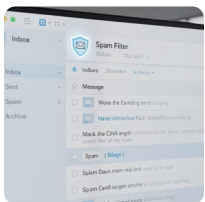
08

Overfitting e Soluzioni

Il problema dell'overfitting e le tecniche pratiche per risolverlo: più dati, feature selection e regolarizzazione

Che cos'è la Classificazione?

La classificazione è un tipo di problema di Machine Learning in cui l'obiettivo è predire a quale categoria appartiene un'osservazione. A differenza della regressione, dove l'output è un valore continuo (come un prezzo o una temperatura), nella classificazione l'output **y** può assumere solo **valori discreti** che rappresentano categorie o classi specifiche.



Filtro Spam

Questa email è spam?

- Classe 0: Email legittima
- Classe 1: Spam



Frodi Finanziarie

La transazione è fraudolenta?

- Classe 0: Transazione legittima
- Classe 1: Frode



Diagnosi Medica

Il tumore è maligno?

- Classe 0: Benigno
- Classe 1: Maligno

📌 **ATTENZIONE:** I termini "negativo" ($y=0$) e "positivo" ($y=1$) sono convenzioni matematiche e NON significano "cattivo" e "buono"! Rappresentano semplicemente l'assenza o la presenza della caratteristica che stiamo cercando di identificare.

Il Dataset di Esempio: Tumori

Training set

	tumor size (cm) x_0	...	patient's age x_1	malignant? y
$i = 1$	10		52	1
$\vdots$	2		73	0
$\vdots$	5		55	0
$i = m$	12		49	1
	...		...	...

$i = 1, \dots, m$ training examples
 $j = 1, \dots, n$ features
target y is 0 or 1

Per comprendere concretamente come funziona la classificazione, utilizzeremo un dataset medico reale che contiene informazioni su pazienti con tumori. Questo è un classico esempio di **Training Set** per la classificazione binaria, dove l'obiettivo è imparare a distinguere tra tumori benigni e maligni basandosi su caratteristiche misurabili.

Features (x)

Le variabili di input che usiamo per fare previsioni:

- **Dimensione del tumore:** misurata in centimetri o millimetri
- **Età del paziente:** un fattore di rischio importante
- Altre caratteristiche cliniche rilevanti

Target (y)

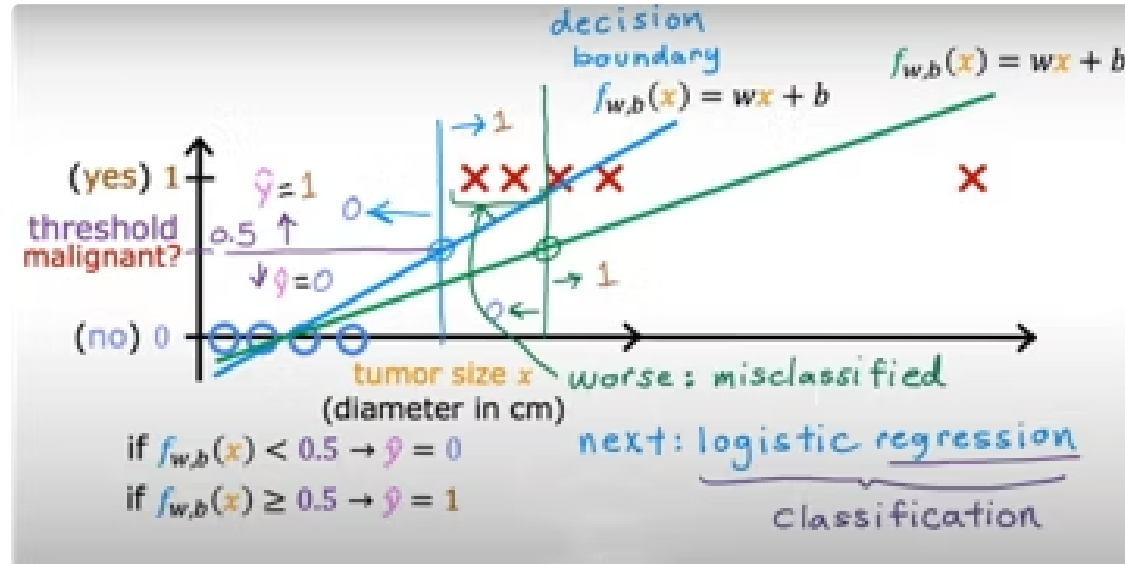
La variabile che vogliamo predire rappresenta la diagnosi:

- **y = 0:** Tumore benigno (non canceroso)
- **y = 1:** Tumore maligno (canceroso)

Il modello imparerà dai dati storici etichettati per fare previsioni accurate su nuovi pazienti.

Perché la Regressione Lineare NON Funziona

Quando affrontiamo un problema di classificazione, potremmo essere tentati di usare una semplice regressione lineare con la formula $f(x) = wx + b$. Tuttavia, questo approccio presenta problemi fondamentali che lo rendono inadatto per la classificazione.



⚠ Apparente Funzionalità

Inizialmente, fissando una soglia a 0.5 per separare le classi, il modello lineare sembra funzionare ragionevolmente bene sui dati di training.

📌 Problema degli Outlier

Basta un solo dato anomalo o estremo per spostare significativamente la retta di regressione, causando errori di classificazione su punti che prima erano correttamente classificati.

📌 Valori Impossibili

La retta può predire valori maggiori di 1 o minori di 0, che non hanno alcun senso quando interpretiamo l'output come una probabilità (le probabilità devono essere comprese tra 0 e 1).

Conclusione: Serve un modello diverso che produca sempre output compresi tra 0 e 1 e che sia più robusto agli outlier. La soluzione è la [Regressione Logistica](#).

Laboratorio: Regressione Lineare per la Classificazione

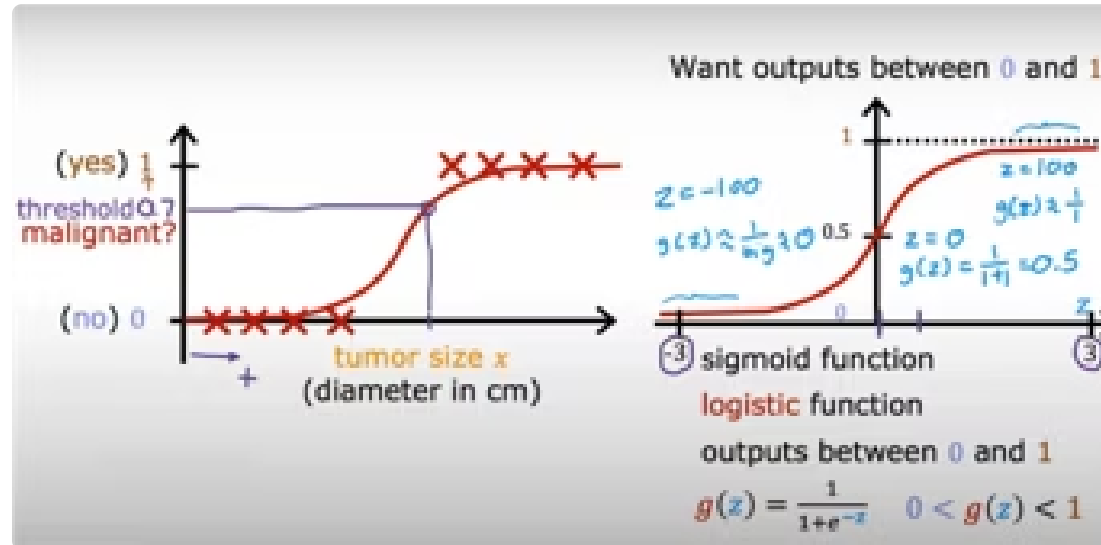
Per illustrare concretamente i limiti della regressione lineare nei problemi di classificazione, ti guideremo attraverso un `notebook` interattivo. Questa sessione pratica ti permetterà di visualizzare come questo approccio sia inadeguato e perché necessitiamo di un modello più sofisticato.

Esamineremo come la regressione lineare, pur producendo una retta, non sia in grado di generare probabilità significative o di gestire in modo robusto la presenza di outlier, portando a conclusioni errate per il nostro dataset di tumori.

- ❏ Preparati a mettere in pratica le nozioni apprese e a osservare in prima persona le sfide che la regressione lineare incontra quando applicata alla classificazione binaria.

La Funzione Sigmoidale (Logistic Function)

La funzione sigmoide, anche chiamata funzione logistica, è il componente matematico chiave che rende possibile la regressione logistica. Questa elegante funzione trasforma qualsiasi valore reale in un output compreso tra 0 e 1, perfetto per rappresentare probabilità.



La Formula

$$g(z) = \frac{1}{1 + e^{-z}}$$

Dove z può essere qualsiasi numero reale (da $-\infty$ a $+\infty$), ed e è il numero di Nepero (≈ 2.718).

Proprietà Fondamentali

- **Output limitato:** Il risultato è sempre compreso tra 0 e 1, perfetto per le probabilità
- **Comportamento asintotico:** Quando $z \rightarrow +\infty$, $g(z) \rightarrow 1$; quando $z \rightarrow -\infty$, $g(z) \rightarrow 0$
- **Punto centrale:** Quando $z = 0$, $g(z) = 0.5$ esattamente
- **Forma a "S":** La curva ha una caratteristica forma sigmoide che transita dolcemente da 0 a 1

Costruire il Modello

La regressione logistica combina l'approccio lineare che già conosciamo con la funzione sigmoide, creando un modello potente ma interpretabile. Il processo avviene in due fasi sequenziali che trasformano le features di input in una probabilità di output.

1

Passo 1: Combinazione Lineare

Calcolare la combinazione lineare delle features, esattamente come nella regressione lineare:

$$z = w \cdot x + b$$

Dove **w** sono i pesi, **x** le features e **b** il bias.

2

Passo 2: Applicare la Sigmoide

Applicare la funzione sigmoide al risultato per ottenere una probabilità:

$$f(x) = g(z) = \frac{1}{1 + e^{-z}}$$

Questo trasforma z in un valore tra 0 e 1.

Interpretation of logistic regression output

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

"probability" that class is 1

Example:

x is "tumor size"
y is 0 (not malignant)
or 1 (malignant)

$f_{\vec{w},b}(\vec{x}) = 0.7$
70% chance that **y** is 1

$$f_{\vec{w},b}(\vec{x}) = P(\mathbf{y} = 1 | \vec{x}; \vec{w}, b)$$

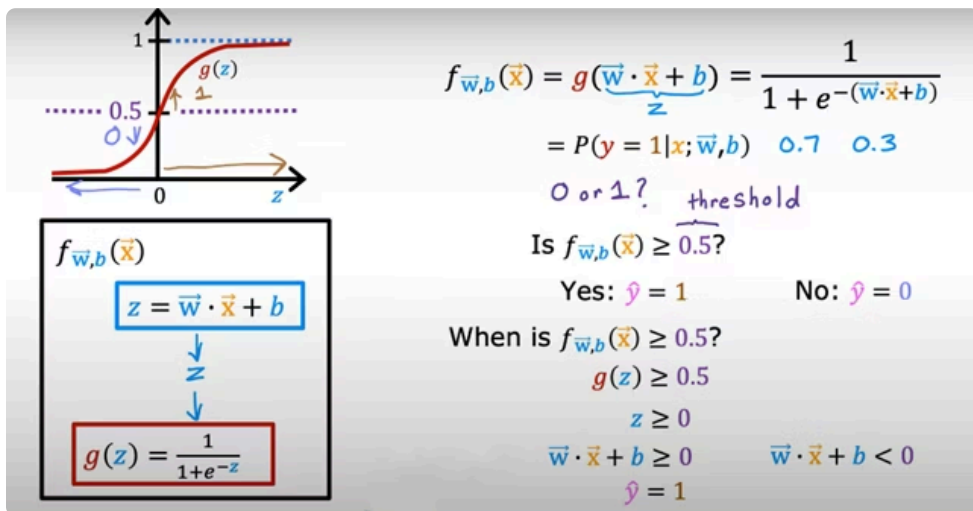
Probability that **y** is 1,
given input **x**, parameters **w**, **b**

$$P(\mathbf{y} = 0) + P(\mathbf{y} = 1) = 1$$

❏ **Interpretazione del Risultato:** L'output $f(x)$ rappresenta la probabilità che $y=1$ dato l'input x . Ad esempio, se $f(x) = 0.7$, significa che c'è il **70% di probabilità** che il tumore sia maligno ($y=1$). In notazione matematica: $P(y=1 | x; w, b)$

Interpretazione Probabilistica

Uno degli aspetti più potenti della regressione logistica è che il suo output può essere interpretato direttamente come una probabilità. Questo rende il modello non solo predittivo, ma anche informativo sulla certezza delle sue previsioni.



Come Funziona

L'output $f(x)$ rappresenta $P(y=1 | x)$, cioè la probabilità che $y=1$ condizionata ai valori delle features x .

Poiché le probabilità devono sommare a 1:

$$P(y = 0) + P(y = 1) = 1$$

Se $f(x) = 0.7$, automaticamente $P(y=0) = 0.3$

Esempio Pratico: Diagnosi Tumore

Se per un paziente il modello calcola $f(x) = 0.7$:

- **70% di probabilità** che il tumore sia maligno ($y=1$)
- **30% di probabilità** che il tumore sia benigno ($y=0$)

Prendere la Decisione

Tipicamente, usiamo una **soglia di decisione** a 0.5:

- Se $f(x) \geq 0.5 \rightarrow$ prediciamo $y=1$ (positivo)
- Se $f(x) < 0.5 \rightarrow$ prediciamo $y=0$ (negativo)

La soglia può essere adattata in base al contesto applicativo.

Esempio Numerico: w e b in Azione

Per capire concretamente come i parametri w e b influenzano le previsioni del modello, analizziamo un esempio numerico con un singolo feature. Consideriamo un modello semplice dove $w = 1$ e $b = -3$, quindi la nostra funzione diventa: $z = x - 3$

Caso 1: $x = 1$

$$z = 1 - 3 = -2$$

$$g(-2) \approx 0.12$$

Previsione: $\hat{y} = 0$

(Tumore Benigno)

Il modello è abbastanza sicuro che sia benigno.

1

2

3

Caso 3: $x = 5$

$$z = 5 - 3 = 2$$

$$g(2) \approx 0.88$$

Previsione: $\hat{y} = 1$

(Tumore Maligno)

Il modello è abbastanza sicuro che sia maligno.

Caso 2: $x = 3$

$$z = 3 - 3 = 0$$

$$g(0) = 0.5$$

Soglia di Incertezza

Il modello è completamente incerto. Siamo esattamente al confine tra le due classi (decision boundary).

Osservazione importante: Il valore di z determina quanto siamo "sicuri" della classificazione. Valori di z molto negativi o molto positivi indicano alta confidenza, mentre valori vicini a zero indicano incertezza.

Laboratorio: Esplorando la Funzione Sigmoidale

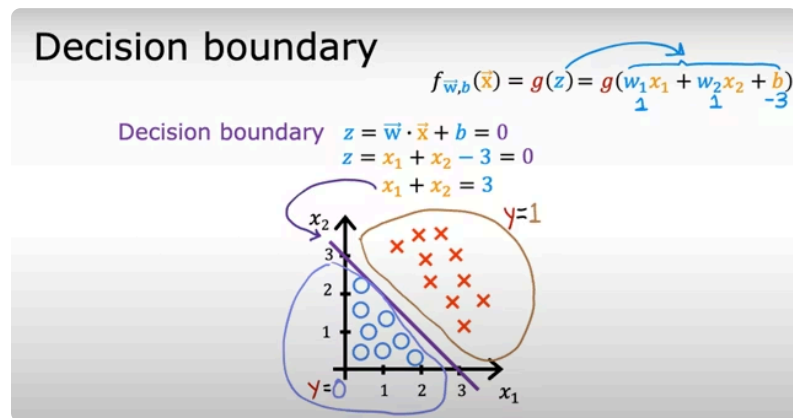
Ora che abbiamo compreso la teoria dietro la funzione sigmoide e come essa trasforma i valori lineari in probabilità, è il momento di vederla in azione! In questo laboratorio interattivo, avrai l'opportunità di manipolare direttamente i parametri e osservare come influenzano la curva sigmoide.

Esploreremo visivamente come la funzione sigmoide mappa i valori di input (z) su un intervallo da 0 a 1, e come la forma a "S" della curva sia ideale per la classificazione, permettendo al modello di esprimere la "fiducia" nelle sue previsioni.

📄 Mettiti comodo e preparati a sperimentare! Capire la sigmoide è fondamentale per padroneggiare la Regressione Logistica.

Il Decision Boundary

Il decision boundary (confine decisionale) è la linea, superficie o ipersuperficie che separa le regioni dello spazio delle features in cui il modello predice classi diverse. È un concetto fondamentale per visualizzare e comprendere come il modello prende le sue decisioni.



Definizione Matematica

Il decision boundary si trova dove $f(x) = 0.5$, che corrisponde al punto in cui:

$$z = w \cdot x + b = 0$$

In questo punto, il modello è completamente incerto tra le due classi.

1

Esempio con 2 Features

Consideriamo un modello con due features x_1 e x_2 , e parametri $w_1=1$, $w_2=1$, $b=-3$:

$$z = x_1 + x_2 - 3 = 0$$

2

Equazione della Retta

Risolvendo per il decision boundary otteniamo:

$$x_1 + x_2 = 3$$

Questa è l'equazione di una retta nello spazio 2D.

3

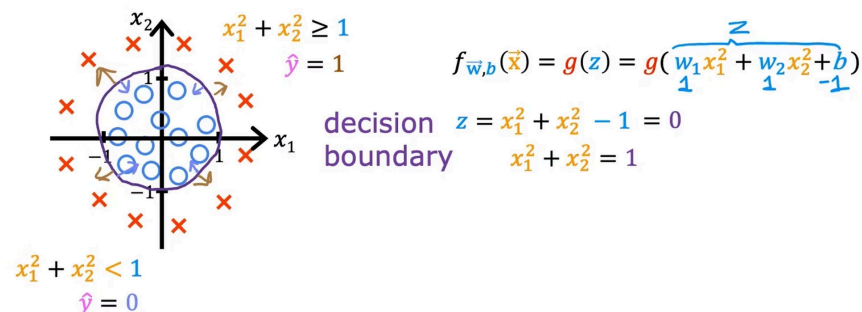
Interpretazione Geometrica

I punti sopra la retta ($x_1 + x_2 > 3$) avranno $\hat{y} = 1$, mentre i punti sotto la retta ($x_1 + x_2 < 3$) avranno $\hat{y} = 0$.

Decision Boundary Non Lineari

La regressione logistica non è limitata a confini decisionali lineari (rette o piani). Quando i dati non sono linearmente separabili, possiamo utilizzare **features polinomiali** per creare decision boundary con forme più complesse che si adattano meglio alla struttura naturale dei dati.

Non-linear decision boundaries



Esempio: Decision Boundary Circolare

Supponiamo di voler separare i punti all'interno di un cerchio da quelli all'esterno. Possiamo usare features polinomiali quadratiche:

$$z = x_1^2 + x_2^2 - 1$$

Il decision boundary si trova dove $z = 0$:

$$x_1^2 + x_2^2 = 1$$

Questa è l'equazione di un **cerchio** con raggio 1 centrato nell'origine!

Come Funziona

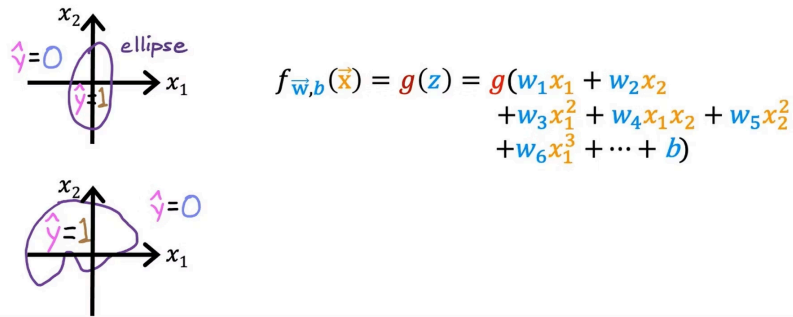
- **Dentro il cerchio:** Per punti dove $x_1^2 + x_2^2 < 1$, avremo $z < 0$, quindi $f(x) < 0.5$ e $\hat{y} = 0$
- **Sul cerchio:** Esattamente $x_1^2 + x_2^2 = 1$, quindi $z = 0$, $f(x) = 0.5$ (incertezza massima)
- **Fuori dal cerchio:** Per punti dove $x_1^2 + x_2^2 \geq 1$, avremo $z \geq 0$, quindi $f(x) \geq 0.5$ e $\hat{y} = 1$

☐ Questo dimostra come la regressione logistica, pur essendo basata su una funzione lineare di z , possa creare confini decisionali non lineari quando usiamo trasformazioni polinomiali delle features originali.

Decision Boundary Complessi

Con features polinomiali di grado superiore, possiamo creare decision boundary estremamente complessi che si adattano a pattern di dati molto articolati. Tuttavia, questa flessibilità ha un costo: il rischio di overfitting.

Non-linear decision boundaries



Polinomi di Alto Grado

Possiamo costruire modelli con termini polinomiali di grado 3, 4 o superiore:

$$f(x) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 + \dots)$$

Forme Possibili

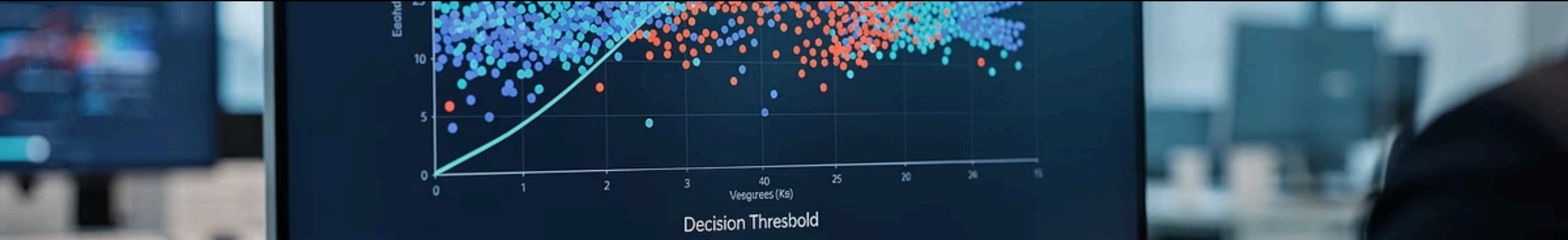
Questo permette di creare decision boundary con forme molto varie:

- Ellissi e parabole
- Forme irregolari e asimmetriche
- Confini con curve multiple
- Regioni di decisione disconnesse

⚠ Il Rischio

Maggiore flessibilità significa anche maggiore rischio di **overfitting**: il modello potrebbe adattarsi troppo bene ai dati di training, inclusi rumore e outliers, perdendo la capacità di generalizzare su nuovi dati.

Principio guida: Il decision boundary dovrebbe essere *abbastanza complesso* da catturare i pattern reali nei dati, ma *abbastanza semplice* da generalizzare bene. Trovare questo equilibrio è una delle sfide chiave del Machine Learning.



Laboratorio: Esplorando i Decision Boundary

Ora che abbiamo esplorato la teoria dei decision boundary lineari e non lineari, è tempo di metterli alla prova in un ambiente interattivo. Questo laboratorio ti permetterà di visualizzare e manipolare direttamente i confini decisionali, offrendoti una comprensione pratica di come il modello logistico separa le classi.



Confini Lineari

Aggiusta i parametri **w** e **b** per vedere come una semplice retta separa i punti dati.

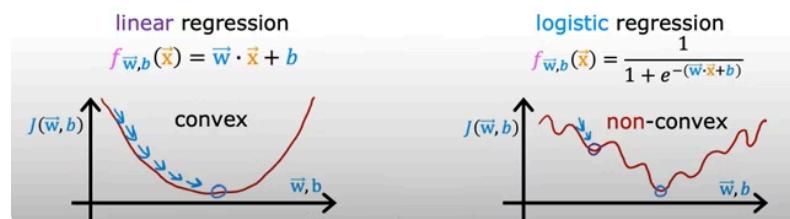
- ☐ L'interazione con i decision boundary è fondamentale per sviluppare un'intuizione solida su come i modelli di classificazione prendono le loro decisioni.

Perché NON Usare l'Errore Quadratico (MSE)

Nella regressione lineare, abbiamo usato l'errore quadratico medio (MSE - Mean Squared Error) come funzione di costo. Tuttavia, per la regressione logistica, l'MSE crea problemi matematici significativi che rendono l'ottimizzazione estremamente difficile o addirittura impossibile.

Squared error cost

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$$



A sinistra: funzione convessa (regressione lineare). A destra: funzione non convessa (logistica con MSE)

Il Problema Fondamentale

Quando combiniamo l'MSE con la funzione sigmoide, otteniamo una funzione di costo **non convessa**, caratterizzata da:

- **Molti minimi locali:** zone in cui la funzione sembra minima, ma non lo è globalmente
- **Plateau e zone piatte:** regioni dove il gradiente è quasi zero
- **Gradient descent instabile:** l'algoritmo può facilmente "bloccarsi" in un minimo locale

Regressione Lineare con MSE

La funzione di costo è **convessa**: ha un solo minimo globale, come una valle o una scodella. Il gradient descent converge sempre al minimo globale.

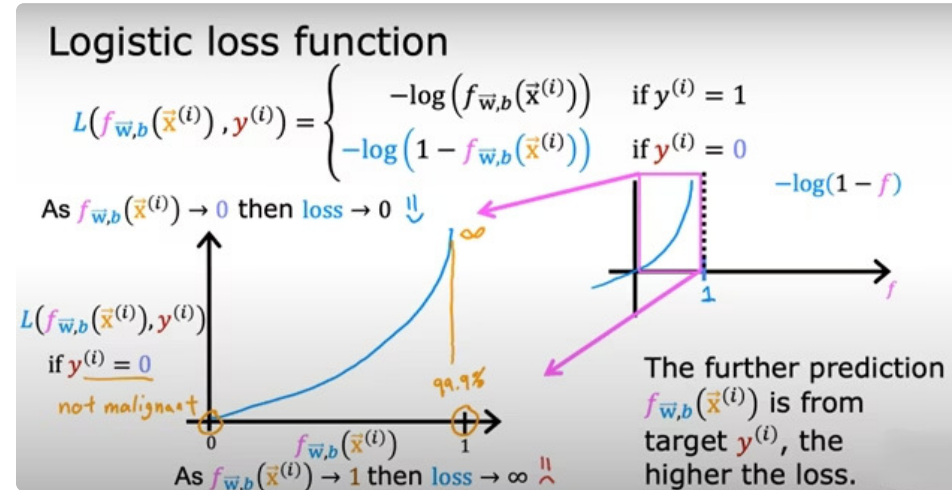
Regressione Logistica con MSE

La funzione diventa **non convessa**: piena di "colline" e "valli" (minimi locali). Il gradient descent può bloccarsi in qualsiasi valle, senza mai trovare il minimo globale vero.

Questo è il motivo per cui abbiamo bisogno di una funzione di costo diversa, progettata specificamente per la classificazione: la **Log Loss** o **Binary Cross-Entropy**.

La Loss Function Logaritmica

La soluzione al problema dell'MSE nella classificazione è una funzione di loss basata sui logaritmi. Questa funzione è progettata specificamente per penalizzare in modo appropriato gli errori di classificazione, mantenendo allo stesso tempo la convessità necessaria per l'ottimizzazione.



Caso 1: $y = 1$ (Classe Positiva)

$$L = -\log(f(x))$$

Comportamento:

- Se $f(x) \rightarrow 1$: la previsione è corretta e sicura. Loss $\rightarrow 0$ (nessuna penalità)
- Se $f(x) \rightarrow 0.5$: la previsione è incerta. Loss moderata
- Se $f(x) \rightarrow 0$: la previsione è sbagliata e sicura. Loss $\rightarrow \infty$ (penalità massima)

Caso 2: $y = 0$ (Classe Negativa)

$$L = -\log(1 - f(x))$$

Comportamento:

- Se $f(x) \rightarrow 0$: la previsione è corretta e sicura. Loss $\rightarrow 0$ (nessuna penalità)
- Se $f(x) \rightarrow 0.5$: la previsione è incerta. Loss moderata
- Se $f(x) \rightarrow 1$: la previsione è sbagliata e sicura. Loss $\rightarrow \infty$ (penalità massima)

Principio chiave: La loss function logaritmica "punisce severamente" le previsioni sbagliate fatte con alta sicurezza. Se il modello è molto sicuro di una previsione ma sbaglia, la penalità è enorme. Se è incerto, la penalità è più moderata.

Cost

$$\begin{aligned} \text{cost} \\ J(\bar{w}, b) &= \frac{1}{m} \sum_{i=1}^m L(\underbrace{f_{\bar{w}, b}(\bar{x}^{(i)})}_{\text{loss}}, y^{(i)}) \\ &= \begin{cases} -\log(f_{\bar{w}, b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\bar{w}, b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases} \end{aligned}$$

Handwritten notes: "convex" and "can reach a global minimum" with an arrow pointing to the loss function.

La Cost Function Completa

La funzione di costo $J(w, b)$ per la regressione logistica è semplicemente la media delle loss individuali calcolate su tutti gli esempi del training set. Questa aggregazione mantiene tutte le proprietà desiderabili della loss logaritmica, in particolare la **convessità**.

Definizione Matematica

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(f(x^{(i)}), y^{(i)})$$

Dove:

- m è il numero totale di esempi nel training set
- L è la loss logaritmica per un singolo esempio
- $f(x^{(i)})$ è la previsione del modello per l'esempio i -esimo
- $y^{(i)}$ è l'etichetta vera dell'esempio i -esimo

Proprietà Fondamentale

Convessità Garantita

Grazie alla scelta della loss logaritmica, questa funzione di costo è convessa: ha un solo minimo globale, con una forma a "scodella".

Il Gradient Descent convergerà sempre al minimo globale!

- ☐ La convessità è una proprietà matematica cruciale: significa che non ci sono "trappole" (minimi locali) in cui l'algoritmo di ottimizzazione possa rimanere bloccato. Qualunque sia il punto di partenza, il gradient descent troverà sempre la soluzione ottimale.

La Formula Semplificata

Le due formule separate per la loss (una per $y=1$ e una per $y=0$) possono essere elegantemente combinate in un'unica espressione matematica. Questa formula compatta è quella che viene effettivamente implementata nel codice ed è più facile da derivare per il gradient descent.

Simplified loss function

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\bar{w},b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\bar{w},b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f_{\bar{w},b}(\bar{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\bar{w},b}(\bar{x}^{(i)}))$$

if $y^{(i)} = 1$: 0 $(1-0)$

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = -1 \log(f(\bar{x}))$$

if $y^{(i)} = 0$:

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = - (1-0) \log(1 - f(\bar{x}))$$

Formula Unificata della Loss

$$L(f(x), y) = -y \log(f(x)) - (1 - y) \log(1 - f(x))$$

Questa singola formula copre entrambi i casi automaticamente, sfruttando il fatto che y può valere solo 0 o 1.

Come Funziona il Trucco

Quando $y = 1$:

- Il termine $(1-y)$ diventa 0
- Il secondo termine si annulla
- Rimane: $L = -\log(f(x))$ ✓

Quando $y = 0$:

- Il termine y diventa 0
- Il primo termine si annulla
- Rimane: $L = -\log(1-f(x))$ ✓

Questa formulazione è particolarmente elegante perché sfrutta le proprietà algebriche dei valori binari (0 e 1) per "spegnere" automaticamente il termine non necessario, senza dover usare condizioni if-then.

La Cost Function Finale

Sostituendo la formula semplificata della loss nella definizione di costo medio, otteniamo l'espressione finale della funzione di costo per la regressione logistica. Questa è la funzione che dobbiamo minimizzare per addestrare il nostro modello.

Simplified cost function

$$\begin{aligned} \text{loss} \\ L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) &= -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \\ \text{cost} \\ J(\vec{w}, b) &= \frac{1}{m} \sum_{i=1}^m [L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})] \\ &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))] \end{aligned}$$

convex
(single global minimum)

Formula Completa

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f(x^{(i)})) + (1 - y^{(i)}) \log(1 - f(x^{(i)}))]$$

Questa è la funzione obiettivo che vogliamo minimizzare rispetto ai parametri w e b .

Nome Tecnico

Questa funzione è conosciuta come:

- **Binary Cross-Entropy Loss**
- **Log Loss**
- **Logistic Loss**

È la funzione di costo standard e universalmente accettata per problemi di classificazione binaria.

Origine Teorica

Questa formula non è arbitraria: deriva dal principio statistico del **Maximum Likelihood Estimation** (MLE). Minimizzare questa funzione equivale a massimizzare la probabilità che il modello generi correttamente le etichette osservate nei dati di training.

Il Gradient Descent

Ora che abbiamo definito la funzione di costo $J(w,b)$, dobbiamo trovare i valori ottimali dei parametri w e b che la minimizzano. Per fare questo, utilizziamo lo stesso algoritmo iterativo che abbiamo già visto nella regressione lineare: il **gradient descent**.

Gradient descent for logistic regression

repeat { *looks like linear regression!*

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right]$$

$$b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$$

} simultaneous updates

Same concepts:

- Monitor gradient descent (learning curve)
- Vectorized implementation
- Feature scaling

Linear regression $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

Logistic regression $f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$

L'Algoritmo

Il gradient descent aggiorna iterativamente i parametri seguendo la direzione opposta al gradiente:

```
Repeat {  
  w_j = w_j - α * ∂/∂w_j J(w,b)  
  b = b - α * ∂/∂b J(w,b)  
}
```

Dove:

- α (alpha) è il learning rate
- $\partial/\partial w_j$ è la derivata parziale rispetto a w_j

🚨 **Importante: Aggiornamento Simultaneo!** Tutti i parametri (tutti i w_j e b) devono essere aggiornati contemporaneamente nella stessa iterazione. Non dobbiamo usare i nuovi valori di alcuni parametri per calcolare gli aggiornamenti di altri parametri nella stessa iterazione.

Le formule specifiche per le derivate parziali della regressione logistica, quando espresse completamente, hanno una forma molto simile a quelle della regressione lineare, anche se derivano da funzioni di costo diverse.

Confronto: Lineare vs Logistica

A prima vista, le regole di aggiornamento del gradient descent per la regressione lineare e la regressione logistica sembrano identiche. Tuttavia, questa somiglianza superficiale nasconde una differenza fondamentale nella definizione della funzione $f(x)$.

La Formula di Aggiornamento (Identica per entrambe)

$$w_j = w_j - \alpha \cdot \frac{1}{m} \cdot \sum_{i=1}^m (f(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Questa è la regola per aggiornare ogni peso w_j . La formula è matematicamente identica sia nella regressione lineare che nella regressione logistica.

MA: $f(x)$ è Definita Diversamente!

Regressione Lineare:

$$f(x) = w \cdot x + b$$

Una semplice combinazione lineare delle features.

Regressione Logistica:

$$f(x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

La combinazione lineare passa attraverso la funzione sigmoide.

Stessa formula, definizione diversa: Questo significa che, anche se le regole di aggiornamento sembrano uguali, il gradient descent sta in realtà ottimizzando funzioni di costo completamente diverse (MSE per la lineare, Binary Cross-Entropy per la logistica) e producendo modelli con comportamenti molto diversi.

Il Processo di Training

Il training di un modello di regressione logistica è un processo iterativo che parte da valori iniziali dei parametri e li raffina progressivamente fino a raggiungere una soluzione ottimale. Vediamo nel dettaglio come funziona questo processo passo dopo passo.



Inizializzazione

Impostiamo i parametri w e b a valori iniziali. Tipicamente si usano zeri o valori casuali piccoli. La scelta dell'inizializzazione può influenzare la velocità di convergenza ma, grazie alla convessità, non il risultato finale.



Calcolo del Gradiente

Per ogni parametro, calcoliamo la derivata parziale della funzione di costo $J(w,b)$. Il gradiente indica la direzione di massima crescita della funzione - vogliamo andare nella direzione opposta per minimizzarla.



Aggiornamento dei Parametri

Modifichiamo simultaneamente tutti i parametri w e b muovendoci nella direzione opposta al gradiente, con un passo proporzionale al learning rate α . Più alto è α , più grandi sono i passi (ma rischio di instabilità).



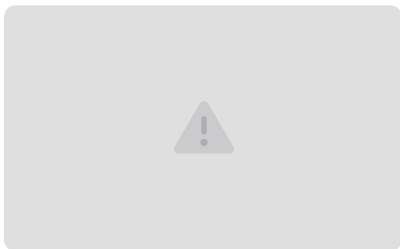
Valutazione e Ripetizione

Ricalcoliamo $J(w,b)$ con i nuovi parametri. Se la funzione di costo continua a scendere significativamente, ripetiamo il processo. Quando $J(w,b)$ smette di diminuire (convergenza), abbiamo trovato i parametri ottimali.

Obiettivo finale: Trovare i valori di w e b che minimizzano la funzione di costo $J(w,b)$, ottenendo così il modello che meglio si adatta ai dati di training e che si spera generalizzi bene su nuovi dati.

Il Problema dell'Overfitting (Regressione)

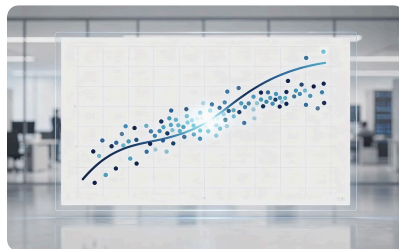
L'overfitting è uno dei problemi più comuni e insidiosi nel Machine Learning. Si verifica quando un modello impara troppo bene i dati di training, inclusi rumore e anomalie, perdendo la capacità di generalizzare su nuovi dati. Vediamo come si manifesta nei problemi di regressione.



Underfitting (High Bias)

Il modello è troppo semplice per catturare la complessità dei dati. Ad esempio, usare una retta per dati che seguono una curva.

Sintomo: Errore alto sia nel training set che nel test set.



Just Right (Generalization)

Il modello cattura il trend generale dei dati senza inseguire ogni singolo punto o anomalia.

Sintomo: Buone performance sia sul training che sul test set. Il modello generalizza bene.



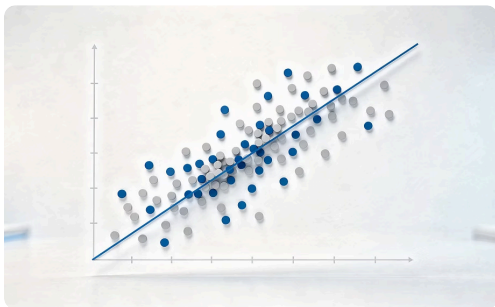
Overfitting (High Variance)

Il modello è troppo complesso (es. polinomio di grado molto alto) e passa esattamente per ogni punto di training, imparando anche il rumore.

Sintomo: Errore bassissimo nel training, ma altissimo nel test.

L'Overfitting nella Classificazione

Il concetto di overfitting si applica anche ai problemi di classificazione, dove si manifesta attraverso la forma del decision boundary. Un confine decisionale troppo complesso cerca di classificare perfettamente ogni singolo punto di training, anche quelli che potrebbero essere outliers o rumore.



Underfitting (High Bias)

Il decision boundary è una semplice retta o un confine troppo semplice che non riesce a separare adeguatamente le classi.

Problema: Molti punti vengono classificati erroneamente, anche nel training set.



Just Right (Generalization)

Il decision boundary è una curva ragionevole che separa la maggior parte dei dati, ignorando outliers e rumore.

Vantaggio: Classificazione accurata sia sui dati di training che su nuovi dati mai visti.



Overfitting (High Variance)

Il confine è estremamente contorto e irregolare per classificare perfettamente ogni singolo punto di training.

Problema: Non generalizzerà su nuovi dati perché ha imparato il rumore invece del pattern reale.

Come Affrontare l'Overfitting

Quando il nostro modello presenta alta varianza (overfitting), abbiamo diverse strategie a disposizione per migliorare la sua capacità di generalizzazione. La scelta della strategia migliore dipende dal contesto specifico del problema e dalle risorse disponibili.

1

Raccogliere Più Dati di Training

Aumentare il numero di esempi nel training set (m) è spesso il modo più efficace per ridurre l'overfitting. Più dati costringono il modello a imparare il pattern generale invece di memorizzare casi specifici o rumore.

Vantaggio: Non richiede modifiche al modello.

Limite: Ottenere più dati può essere costoso, lento o impossibile.

2

Feature Selection

Ridurre il numero di features in ingresso, manualmente (usando la conoscenza del dominio) o automaticamente (usando algoritmi di Model Selection). Meno features significano meno complessità e meno rischio di overfitting.

Vantaggio: Modello più semplice e interpretabile.

Limite: Rischiamo di scartare informazioni utili.

3

Regolarizzazione

Mantenere tutte le features, ma aggiungere una penalità alla funzione di costo che scoraggia valori grandi dei parametri w . Questo forza il modello a essere più semplice senza eliminare completamente nessuna feature.

Vantaggio: Mantiene tutte le informazioni, bilanciando fit e semplicità.

Flessibilità: Il parametro λ controlla quanto vogliamo semplificare.

Strategia 1: Più Dati

L'approccio più diretto e spesso più efficace per combattere l'overfitting è aumentare la dimensione del training set. Quando un modello ha accesso a più esempi diversi, diventa più difficile per esso "memorizzare" i dati invece di imparare i pattern generali.

Perché Funziona

Un modello che "memorizza" il rumore e gli outliers presenti in un piccolo training set non riuscirà a adattarsi altrettanto bene quando viene esposto a una grande massa di nuovi punti con variazioni naturali.

Più dati costringono il modello a identificare e imparare il **pattern sottostante reale**, quello che è consistente attraverso molti esempi diversi, ignorando le peculiarità e il rumore di singoli punti.

In pratica, con più dati diventa statisticamente impossibile per il modello ottenere un fit perfetto su tutti i punti mantenendo alta complessità - deve necessariamente semplificare e generalizzare.

Il Limite Pratico

Purtroppo, raccogliere più dati non è sempre possibile o conveniente:

- **Costo:** Raccogliere e etichettare nuovi dati può richiedere risorse significative (tempo, denaro, personale)
- **Disponibilità:** In alcuni domini (es. malattie rare, eventi rari) i dati semplicemente non esistono in grandi quantità
- **Privacy:** Regolamenti sulla privacy possono limitare l'accesso a certi tipi di dati
- **Tempo:** La raccolta dati può richiedere mesi o anni

❏ **Regola pratica:** Se hai la possibilità di ottenere più dati di qualità, è quasi sempre la prima strategia da tentare. Quando questo non è possibile, allora considera le altre tecniche come feature selection o regolarizzazione.

Strategia 2: Feature Selection

Quando abbiamo troppe features rispetto al numero di esempi di training, il modello ha troppi gradi di libertà e rischia di overfittare. La feature selection consiste nel selezionare solo le variabili più rilevanti, riducendo la complessità del modello senza perdere (troppo) potere predittivo.

Selezione Manuale

Usiamo la nostra **conoscenza del dominio** e l'esperienza per decidere quali variabili sono realmente importanti per il problema e quali possono essere scartate.

Esempio: In un problema di diagnosi medica, potremmo sapere che certe misurazioni cliniche sono molto predittive mentre altre sono poco correlate alla malattia.

Vantaggio: Incorpora expertise umana e intuizione.

Limite: Richiede competenza nel dominio specifico.

Selezione Automatica

Usiamo **algoritmi di Model Selection** che valutano matematicamente l'importanza di ogni feature e identificano automaticamente quali contribuiscono di più alla predizione.

Tecniche comuni: Forward selection, backward elimination, recursive feature elimination, LASSO.

Vantaggio: Oggettivo e non richiede expertise del dominio.

Limite: Può richiedere molto tempo computazionale.

📄 ⚖️ **Il Trade-off:** La feature selection comporta sempre un rischio: potremmo scartare informazioni che sembrano poco utili individualmente ma che in combinazione con altre features potrebbero essere preziose. È un bilanciamento tra ridurre la complessità e mantenere informazione utile.

La Regularizzazione

La regularizzazione è una tecnica elegante che affronta l'overfitting senza eliminare features: invece di scegliere quali variabili tenere o scartare, manteniamo tutte le features ma penalizziamo la complessità del modello scoraggiando valori grandi dei parametri w .

Come Funziona

Modifichiamo la funzione di costo aggiungendo un termine di **penalità** che cresce con la grandezza dei parametri:

$$J(w, b) = \text{Loss} + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Dove:

- **Loss:** La funzione di costo originale (Binary Cross-Entropy per la logistica)
- **λ (Lambda):** Il parametro di regularizzazione che controlla quanto penalizzare
- **$\sum w_j^2$:** La somma dei quadrati di tutti i pesi (escludendo tipicamente b)

Il Principio

Parametri w piccoli → Modello più semplice

Modello più semplice → Meno overfitting

La penalità forza il modello a "giustificare" valori grandi di w con un miglioramento significativo del fit.

λ Alto

Penalità forte → parametri w molto piccoli → modello molto semplice

Rischio: Underfitting (il modello diventa troppo semplice)

λ Ottimale

Equilibrio perfetto tra fit ai dati e semplicità del modello

Obiettivo: Massimizzare la generalizzazione

λ Basso

Penalità debole → parametri w liberi di crescere → modello complesso

Rischio: Overfitting (torniamo al problema originale)

Riepilogo e Concetti Chiave

Abbiamo completato il nostro viaggio attraverso la regressione logistica, uno degli algoritmi fondamentali del Machine Learning per la classificazione. Rivediamo i concetti principali che abbiamo esplorato e che costituiscono la base per comprendere questo potente strumento.

Regressione Logistica

Un modello per la **classificazione binaria** ($y \in \{0,1\}$) che combina una funzione lineare con la **funzione sigmoide** per produrre probabilità comprese tra 0 e 1.

Output: $P(y=1|x)$ - la probabilità che l'esempio appartenga alla classe positiva.

Decision Boundary

La linea, curva o superficie che separa le regioni dello spazio in cui il modello predice classi diverse. Definito matematicamente da $z = w \cdot x + b = 0$.

Può essere lineare o non lineare usando features polinomiali.

Cost Function

Usiamo la **Log Loss** (Binary Cross-Entropy) invece dell'MSE per garantire la **convessità** della funzione di costo, essenziale per la convergenza del gradient descent.

Penalizza severamente previsioni sbagliate fatte con alta confidenza.

Overfitting

Si verifica quando il modello impara il rumore e gli outliers invece del pattern generale, risultando in ottime performance sul training set ma pessime su nuovi dati.

Soluzioni: Raccogliere più dati, Feature Selection, Regolarizzazione con λ .

La regressione logistica è un fondamento essenziale del Machine Learning. La sua semplicità, interpretabilità e efficacia la rendono uno strumento indispensabile per ogni Data Analyst. Ora avete le basi per applicarla a problemi reali di classificazione!